



Today's agenda

- ↳ Binary number generation
- ↳ TreeMap/TreeSet Intro
- ↳ Problem 1
- ↳ Deque Intro
- ↳ Sliding window maximum



AlgoPrep



Q) Binary number generation

↳ Given a number N , write a function that generates and prints all binary number with decimal values from 1 to N .

$N=3$: 1 10 11

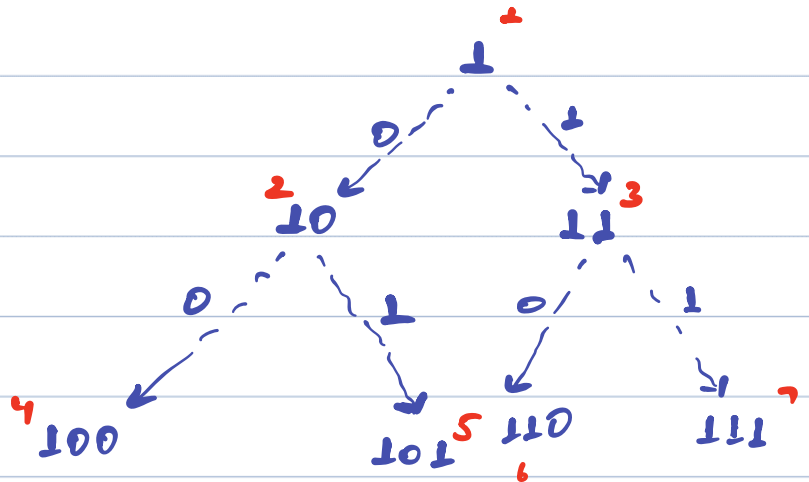
$N=5$: 1 10 11 100 101

Idea 1

↳ iterate for every number and get the binary representation of them 1 by 1.

T.C: $O(N * \log N)$

Idea 2



queue:

~~"1"~~ ~~"10"~~ ~~"11"~~ ~~"100"~~ ~~"101"~~ ~~"110"~~ ~~"111"~~ ~~"1000"~~ ~~"1001"~~ ~~"1010"~~ ~~"1011"~~ ~~"1100"~~ ~~"1101"~~ ~~"1110"~~ ~~"1111"~~

1 10 11 100 101



11PSuedo code

```
void generate Binary (int n) {
```

```
    Queue < String > q;
```

```
    q.add("1");
```

```
    while (n-- > 0) {
```

```
        String rem = q.remove();
```

```
        S.O.P(rem);
```

```
        q.add(rem + "0");
```

```
        q.add(rem + "1");
```

```
    }
```

```
}
```



ordered dict
{Python}

Treemap / Treemap

↳ Sorted Hashmap / Hashset.

↳ keys

Ex: Store Countries and its Population

Country	Population.
Australia	80
India	140
Pakistan	200
USA	50

Syntax:

Treemap < String, Integer > map = new Treemap<>();

map.put("India, 50);
map.containsKey();
map.remove();

→ $O(\log N)$

$O(\log N)$ ←

Ceil(k): Just greater than or equal to k .

Floor(k): Just smaller than or equal to k .



Type 1: i : flip data at i th index.

ex: arr[10]: { 0 0 ~~0~~ 0 0 0 0 ~~0~~ 0 ~~0~~ 0 }

1:2

1:8

1:6

2 : 5 : 6

 $2 \div 4 = 2$

1:6

2:5:2

→ 0 (L)

↳ Tyle 1: replace ith index with $1 - arr[i]$.

~~ans 150~~

$$1 - 0 = 1$$

~~ans[i] = 1~~

$$1-1=0$$

↳ Type 2: iterate to left side as well as right side, return the smaller distance.

$\hookrightarrow O(n)$



// optimal approach

$\rightarrow O(1) \rightarrow O(\log N)$

↳ Type 1: replace ith index with $1 - arr[i]$.

and add the index to TreeSet.

$$arr[i] = 0$$

$$1 - 0 = 1$$

$$arr[i] = 1$$

$$1 - 1 = 0$$

$arr[10]: \{ \overset{0}{0} \overset{1}{0} \overset{2}{1} \overset{3}{0} \overset{4}{0} \overset{5}{0} \overset{6}{1} \overset{7}{0} \overset{8}{1} \overset{9}{0} \}$

$TreeSet: \{ 2 \ 6 \ 8 \}$

1: 2

1: 8

1: 6

2: 5

2: 4

$\rightarrow O(\log N)$

Type 2: $i = 5$

$$\hookrightarrow \text{leftnearest}(5) = \text{floor}(5) = 2$$

$$\hookrightarrow \text{rightnearest}(5) = \text{ceil}(5) = 6$$

$\hookrightarrow 6$

Type 2: $i = 4$

$$\text{leftnearest}(4) = \text{floor}(4) = 2$$

$$\text{rightnearest}(4) = \text{ceil}(4) = 6$$

$\hookrightarrow 2$



11PSuedo Code

```
class Noname {  
    int [] arr;  
    TreeSet < Integer > set;  
    Noname (int n) {  
        arr = new int [n];  
        set = new TreeSet < > ();  
    }  
    void Type 1 (int i) {  
        if (arr[i] == 0) {  
            arr[i] = 1;  
            set.add(i);  
        }  
        else {  
            arr[i] = 0;  
            set.remove(i);  
        }  
    }  
    int Type 2 (int i) {  
        int rightnearest = set.ceiling(i);  
        int leftnearest = set.floor(i);  
        int leftdist = i - leftnearest;  
        int rightdist = rightnearest - i;  
        if (leftdist <= rightdist) {  
            return leftnearest;  
        }  
        else {  
            return rightnearest;  
        }  
    }  
}
```

o(log n) ←

o(log n) ←



// Deque Intro

↳ Double ended queue.



Syntax:

↳ `Deque<Integer> dq = new ArrayDeque<>();`

→ `dq.addFirst(10);`

→ `dq.addLast(20);`

→ `dq.removeFirst();`

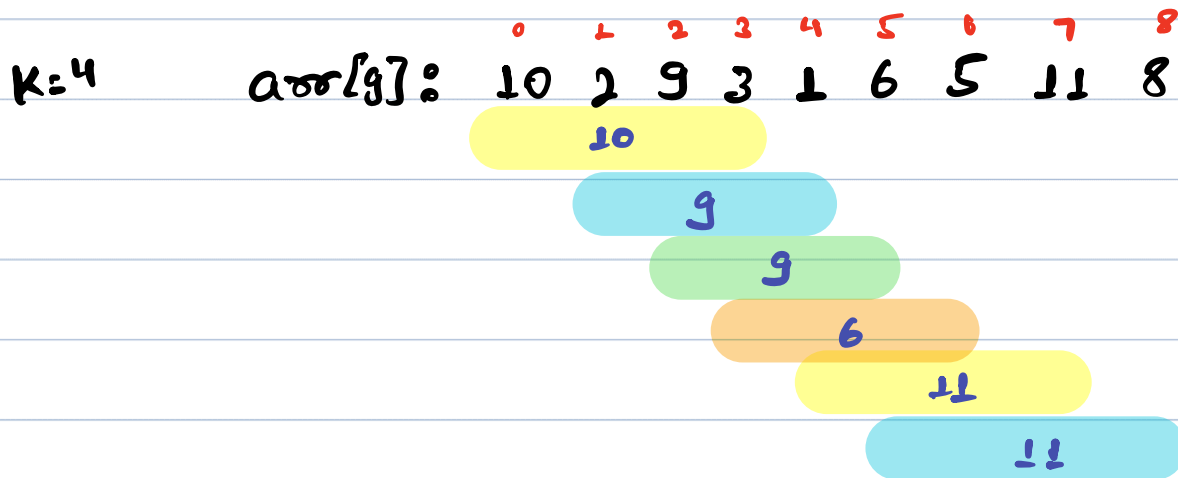
→ `dq.removeLast();`

→ 0 (1)



Q) Sliding window minimum

↳ Given $arr[n]$ and K , find min element in every window of size K .



Q) How many subarray of length K in N length array?
↳ $N-K+1$

Idea 1

↳ for every window of size K , iterate and get min.

T.C: $(N-K+1) * K$ S.C: $O(1)$

$\downarrow$

$N * K - K + K$

$\downarrow$

$O(N * K)$



→ incorrect idea

Idea 2 → Slide the window with max variable.

$k=4$
arr[]: 10 2 9 3 1 6 5 11 8

max = 10

Idea 3

$k=4$
arr[]: 10 2 9 3 1 6 5 11 8

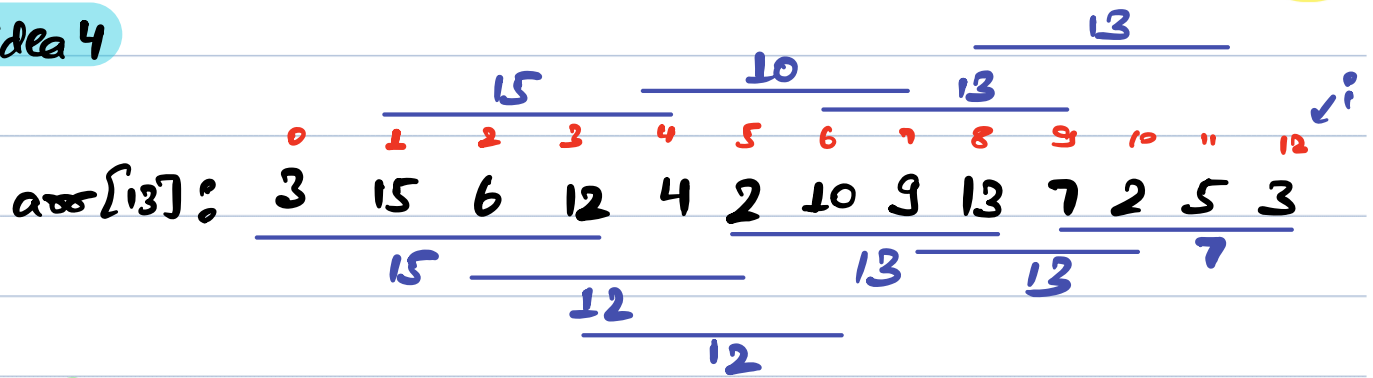
1
~~2~~
3
6
9
~~11~~

T.C: $O((N-k+1) * (\log k)) \approx O(N \log k)$

~~8~~
6
3
2
1



Idea 4



dg:

~~3~~ ~~15~~ ~~12~~ ~~4~~ ~~2~~ ~~10~~ ~~9~~ ~~13~~ 7 ~~2~~ 5 3

if (`arr[i] > dg.getLast()`) { $\rightarrow$

remove from end of dg

12
10

3



AlgoPrep



// Pseudo code

```
void SlidingWindow (int arr[], int k) {  
    Dequeue < Integer > dq;
```

```
    for (int i=0; i<n; i++) {
```

```
        while (dq.size() > 0 && arr[i] > dq.getLast())  
            dq.removeLast();
```

```
    }
```

```
        dq.addLast (arr[i]);
```

```
    }
```

```
    Print (dq.getFirst());
```

```
    for (int i=k; i<n; i++) {
```

```
        // insert ith elem, (i-k)th removed.
```

```
        while (dq.size() > 0 && arr[i] > dq.getLast())  
            dq.removeLast();
```

```
    }
```

```
        dq.addLast (arr[i]);
```

```
        if (dq.getFirst() == arr[i-k]) {  
            dq.removeFirst();
```

```
    }
```

```
    Print (dq.getFirst());
```

```
}
```

3

T.C: $O(2N) \approx O(N)$

S.C: $O(N)$